

Visão Geral

Esta documentação detalha a implementação do firmware desenvolvido em MicroPython para o microcontrolador ESP32 do **Sistema Inteligente de Gestão Hídrica (SIGH)**. O código integra rotinas de sensoriamento analógico, controle digital de atuadores, comunicação de rede sem fio (Wi-Fi), arquitetura de mensageria assíncrona (MQTT) e interface de visualização local (I2C).

1. Estrutura e Arquitetura do Sistema

O algoritmo opera sob uma arquitetura concorrente baseada em um ciclo de varredura (*polling*) controlado por tempo e tratamento de interrupções de rede assíncronas via funções de *callback*. O ecossistema é modularizado em cinco subcamadas lógicas:

1. **Camada de Configuração de Redes (Módulos Wi-Fi e MQTT).**
2. **Camada de Abstração e Inicialização de Hardware (GPIO, ADC e I2C).**
3. **Driver Monolítico de Baixo Nível para Display LCD I2C.**
4. **Mecanismo de Conexão e Auto-recuperação (*Keepalive* / *Reconnect*).**
5. **Loop de Varredura Ambiental e Máquina de Estados Local.**

2. Configurações Iniciais e Definição de Variáveis

2.1. Parâmetros de Rede e Broker

- SSID: Identificador do ponto de acesso sem fio local (configurado para a rede simulada "Wokwi-GUEST").
- SENHA: Credencial de segurança de rede (configurada vazia para ambiente público).
- BROKER: Endereço do servidor centralizador de mensagens MQTT ("test.mosquitto.org").
- CLIENT_ID: Identificador de string exclusivo para autenticação de sessão no Broker ("sigh_mackenzie_luiz_2026").

2.2. Árvore de Tópicos MQTT

- TOPICO_UMIDADE (.../umidade): Canal de publicação de telemetria contendo payloads com a leitura analógica tratada.
- TOPICO_COMANDO (.../comando): Canal de subscrição (assinatura) para receber instruções externas de acionamento (ON / OFF).

- **TOPICO_STATUS (.../status):** Canal de telemetria operacional para atualizar os estados lógicos do dispositivo (ONLINE, IRRI_ON, IRRI_OFF).

3. Mapeamento de Hardware e Interfaces

O ecossistema periférico está configurado diretamente nos barramentos internos do microcontrolador ESP32, operando sob o seguinte circuito lógico:

- **Sensor de Umidade (Potenciômetro Homólogo):** Conectado ao pino analógico **GPIO 34**. Está configurado com atenuação de 11 dB (ADC.ATTN_11DB) para ler a faixa de tensão operacional completa (0 V a 3.3 V), operando em uma escala de quantização discreta de 12 bits (faixa dinâmica de 0 a 4095).
- **Atuador Hídrico (Módulo Válvula / Relé):** Associado à porta digital **GPIO 5** configurada em modo de saída purista (Pin.OUT). O circuito adota lógica invertida: nível lógico baixo (0 ou 0 V) liga os irrigadores; nível lógico alto (1 ou 3.3 V) desativa a bobina de atuação.
- **Display LCD 16x2:** Conectado ao barramento de hardware **I2C(0)** por meio da **GPIO 22 (SCL)** e **GPIO 21 (SDA)**, operando em uma frequência estável de relógio de 400 kHz .

4. Dicionário de Funções e Sub-rotinas

4.1. Funções de Controle do Display LCD I2C (Driver Nativo)

Como o código implementa a manipulação direta do registrador de expansão PCF8574 sem dependência de bibliotecas compiladas complexas, o controle baseia-se no particionamento de nibbles (grupos de 4 bits):

- **lcd_write(data):** Envia um byte de comando ou dado ao barramento físico I2C, preservando o bit de controle da luz de fundo (BACKLIGHT = $0x08$).
- **lcd_toggle(data):** Comuta o pino de habilitação (ENABLE = $0x04$) gerando um pulso quadrado síncrono por hardware para que o display registre as informações.
- **lcd_send_nibble(nibble):** Envia a metade mais significativa de um byte isoladamente para os barramentos de controle de dados em modo de 4 bits.
- **lcd_send_byte(byte, mode):** Divide um caractere de 8 bits em dois nibbles distintos (superior e inferior), combinando-os com o bit de controle de modo (0 para instruções de comando, 1 para caracteres de texto).

- `lcd_cmd(cmd) / lcd_data(data)`: Encapsulam o envio de comandos estruturais do sistema (limpeza, posicionamento) e caracteres alfanuméricos, respectivamente.
- `lcd_init()`: Rotina sequencial obrigatória que rege o protocolo de inicialização por hardware do display LCD em barramento de 4 bits.
- `lcd_clear()`: Envia a instrução de limpeza de memória do registrador de vídeo, zerando o display.
- `lcd_set_cursor(col, row)`: Modifica o endereço DDRAM do display para posicionar o ponteiro de escrita na coluna e linha (0 para linha 1, 1 para linha 2) desejadas.
- `lcd_print(text)`: Varre iterativamente uma string de texto, convertendo cada caractere em sua representação inteira correspondente na tabela ASCII (`ord(c)`) e imprimindo-a na tela.

4.2. Gerenciamento e Conectividade de Rede

- `conecta_wifi()`: Ativa o circuito de radiofrequência em modo Station (STA_IF), realiza o escaneamento do ponto de acesso local e dispara a requisição de conexão. Implementa uma contagem de segurança de 20 segundos (*timeout*); caso obtenha êxito, imprime as configurações IP na tela e no console local.
- `verifica_wifi()`: Executa uma checagem de integridade em tempo de execução. Se o enlace de rádio for rompido por instabilidade ambiental, reinoca automaticamente a rotina `conecta_wifi()` para mitigar quedas de conexão.
- `conecta_mqtt()`: Instancia o cliente da biblioteca `umqtt.simple`, ancora a função de escuta ativa de mensagens remota, realiza a conexão ao Broker público Mosquitto e executa o *subscribe* no tópico de comandos. Ao finalizar, publica a flag "ONLINE" no tópico de status, notificando os servidores centrais de IA.
- `receber_msg(topico, msg)`: Função de *callback* assíncrona engatada ao receptor da pilha TCP/IP. É disparada imediatamente quando o Broker envia uma mensagem no tópico de controle. Caso o payload seja b"ON", chaveia a GPIO 5 para zero (ligando a válvula); se for b"OFF", comuta a GPIO 5 para nível alto (desligando a válvula).
- `verifica_mensagens()`: Invoca o método de verificação de pacotes na fila do cliente MQTT (`client.check_msg()`). Envolve a rotina em um bloco de tratamento de exceções (*try/except*) para interceptar falhas críticas de rede,

forçando uma desconexão limpa e disparando o processo de reconexão automática caso o link com o servidor caia.

4.3. Algoritmo de Processamento do Sensor

- `ler_umidade()`: Executa a leitura imediata do canal analógico 34. Converte o sinal escalonado em valores de tensão e aplica uma função de transferência de mapeamento linear baseada nas constantes calibradas de laboratório:

$$\text{Umidade (\%)} = \frac{\text{Leitura ADC} - \text{MOLHADO}}{\text{SECO} - \text{MOLHADO}} \times 100$$

Garante que o resultado seja truncado rigidamente em formato de inteiro dentro dos limites físicos ideais de segurança de 0% a 100% por meio da instrução composta de contenção `max(0, min(100, int(umidade)))`.

5. Dinâmica Operacional do Loop Principal

Após passar com sucesso pela rotina fixa de *setup* (`lcd_init` → `conecta_wifi` → `conecta_mqtt`), o firmware entra em laço infinito sob a seguinte lógica determinística de tomada de decisão:

1. **Checagem de Link:** Verifica preventivamente a estabilidade do sinal Wi-Fi e faz a varredura ativa por mensagens MQTT na fila de rede.
2. **Leitura e Processamento:** Realiza a amostragem analógica e processa a variável percentual de umidade.
3. **Controle de Autonomia Local (Zona de Estresse Hídrico):**
 - **Se a umidade estiver abaixo do limiar de segurança (UMIDADE_MINIMA = 30%):** O nó de borda assume controle de prioridade local imediata, ativa a GPIO 5 (Válvula ligada), exibe na tela o alerta de ativação e sinaliza a alteração de estado à infraestrutura central via publicação do pacote "IRRI_ON".
 - **Se a umidade estiver acima ou igual ao limite de 30%:** O sistema desliga a válvula (GPIO 5 em nível alto), atualiza as linhas do display local exibindo o percentual exato medido e notifica a rede enviando a string "IRRI_OFF".
4. **Telemetria de Sincronismo:** Independentemente da zona de operação, o valor numérico bruto da umidade do solo é continuamente publicado no tópico MQTT de monitoramento.
5. **Conservação de Banda:** O firmware entra em suspensão controlada de hardware por 10 segundos (`time.sleep(10)`), poupando consumo de rede e

reiniciando o loop de varredura sequencialmente após o término da contagem.